

host_agent

- Initialised

planning_agent

- Initialised

coding_agent

- Initialised

plotting_agent

- Initialised

address_agent

- Initialised

named_area_agent

- Initialised

buildings_agent

- Initialised

water_features_agent

- Initialised

water_network_agent

- Initialised

land_features_agent

-
- Initialised

land_use_features_agent

-
- Initialised

structures_agent

-
- Initialised

human

What is the distance between each house and its closest GP surgery in Exeter?

host_agent

Tool send_message called with args:

```
{'target': 'planning_agent', 'task_description': 'User wants to know the distance between each house and its closest GP surgery in Exeter. Provide the general
```

planning_agent

The general area is Exeter, so we will have 1 entry for Exeter. The entities to search for are houses and GP surgeries within Exeter. We need to find the distance from each house to its closest GP surgery, which involves calculating distances.

output steps: ["Find Exeter 1 area", "Find all houses in Exeter as many search results", "Find all GP surgeries in Exeter as many search results", "Calculate the distance from each house to its closest GP surgery"]

host_agent

Tool send_message returned result: The general area is Exeter, so we will have 1 entry for Exeter. The entities to search for are houses and GP surgeries within Exeter. We need to find the distance from each house to its closest GP surgery, which involves calculating distances.

output steps: ["Find Exeter 1 area", "Find all houses in Exeter as many search results", "Find all GP surgeries in Exeter as many search results", "Calculate the distance from each house to its closest GP surgery"]

Tool generate_metadata_for_all_artifacts called with args: {} Tool generate_metadata_for_all_artifacts returned result: {}

Tool send_message called with args:

```
{'target': 'named_area', 'task_description': 'Find the area polygon for Exeter.'}
```

named_area_agent

Tool call_os_ngd called with args:

```
{'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_polygon'}
```

Tool call_os_ngd returned result: - Search results have been found. Artifact generated is : exeter_polygon, Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter..., count of records fetched : 98.

Tool send_message called with args:

```
{'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'exeter_polygon' to find the specific area polygon for Exeter. Save the filtered re"}
```

data_analysis_agent

Tool generate_metadata_for_artifacts called with args:

```
{'artifact_names': ['exeter_polygon']}
```

Tool generate_metadata_for_artifacts returned result.

Tool Code Executor

```

import matplotlib
matplotlib.use('Agg')
def filter_exeter_polygon(data: list):
    gdf = data[0]
    # Filter for Exeter using multiple relevant columns
    # We look for 'Exeter' in name1_text and also ensure the descriptiongroup and description are relevant
    filtered = gdf[
        (
            (gdf['name1_text'].str.lower() == 'exeter') |
            (gdf['name1_text'].str.lower().str.contains('exeter'))
        ) & (
            gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement', 'Other Name'])
        ) & (
            gdf['description'].isin(['City', 'Town', 'Suburban Area', 'Named Other Geographic Area'])
        )
    ]
    return [
        f"Filtered Exeter polygon(s) found: {len(filtered)} entry(ies).",
        'filtered_exeter_polygon',
        'GeoDataFrame containing only the polygon(s) for Exeter, filtered by name and relevant description columns.',
        filtered
    ]

```

Filtered Exeter polygon(s) found: 1 entry. The result is saved as 'filtered_exeter_polygon', which contains only the polygon(s) for Exeter, filtered by name and relevant description columns. If you need to view or analyze this polygon further, let me know!

Tool send_message returned result: Filtered Exeter polygon(s) found: 1 entry. The result is saved as 'filtered_exeter_polygon', which contains only the polygon(s) for Exeter, filtered by name and relevant description columns. If you need to view or analyze this polygon further, let me know!

named_area_agent

The specific area polygon for Exeter has been successfully filtered and saved as 'filtered_exeter_polygon'. If you need further analysis or details, feel free to ask!

Tool send_message returned result: The specific area polygon for Exeter has been successfully filtered and saved as 'filtered_exeter_polygon'. If you need further analysis or details, feel free to ask!

host_agent

Tool send_message called with args:

```
{'target': 'buildings', 'task_description': "Find all houses in the area defined by 'filtered_exeter_polygon'. Filter for residential houses only."}
```

buildings_agent

Tool call_os_ngd called with args:

```
{'filters': None, 'bbox': 'filtered_exeter_polygon', 'polygon_or_point': True, 'street_address': None, 'filename': 'all_buildings_in_filtered_exeter'}
```

Tool call_os_ngd returned result: - Multiple search results have been found in multiple datasets. A summary of each is provided. Artifacts generated are : ['buildings_all_buildings_in_filtered_exeter', 'buildingline_all_buildings_in_filtered_exeter', 'buildingpart_all_buildings_in_filtered_exeter'], Descriptions are : ["A geopandas dataframe containing building data with filters and bbox applied for the query Find all houses in the area defined by 'filtered_exeter_polygon'. Filter for residential houses only. using filters None.", ...], counts of records fetched are : [69169, 69342, 85480].

host_agent

Tool send_message called with args:

```
{'target': 'data_analysis_agent', 'task_description': "From the artifact 'buildings_all_buildings_in_filtered_exeter', filter and return only the houses as de
```

data_analysis_agent

Tool generate_metadata_for_artifacts called with args:

```
{'artifact_names': ['buildings_all_buildings_in_filtered_exeter']}
```

Tool generate_metadata_for_artifacts returned result.

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_residential_houses_in_filtered_exeter(data: list):
    gdf = data[0]
    filtered = gdf[(gdf['buildinguse_addresscount_residential'] > 0) & (gdf['buildinguse_addresscount_total'] == 1)]
    summary = f"Filtered residential houses: {len(filtered)} entries from {len(gdf)} total buildings."
```

```
artifact_name = 'residential_houses_in_filtered_exeter'
artifact_description = (
    "A GeoDataFrame containing only those buildings in the original dataset where "
    "'buildinguse_addresscount_residential' > 0 and 'buildinguse_addresscount_total' == 1, "
    "representing single-address residential houses in filtered Exeter."
)
return [summary, artifact_name, artifact_description, filtered]
```

Tool Code Executor (first attempt failed due to missing 'data' definition)

Tool Code Executor (second attempt succeeded)

Filtered residential houses: 38,848 entries from 69,169 total buildings.

Output from agent data_analysis_agent: Filtered residential houses: 38,848 entries were found from a total of 69,169 buildings.

An artifact named 'residential_houses_in_filtered_exeter' has been generated. This artifact is a GeoDataFrame containing only those buildings in the original dataset where 'buildinguse_addresscount_residential' > 0 and 'buildinguse_addresscount_total' == 1, representing single-address residential houses in filtered Exeter.

Tool send_message returned result: Filtered residential houses: 38,848 entries were found from a total of 69,169 buildings.

An artifact named 'residential_houses_in_filtered_exeter' has been generated. This artifact is a GeoDataFrame containing only those buildings in the original dataset where 'buildinguse_addresscount_residential' > 0 and 'buildinguse_addresscount_total' == 1, representing single-address residential houses in filtered Exeter.

buildings_agent

The search for all houses in the area defined by 'filtered_exeter_polygon', filtered for residential houses only, has been completed.

Definition used for "house" (as per guidelines): - buildinguse_addresscount_residential > 0 - buildinguse_addresscount_total = 1 (This ensures only single-address residential houses are included.)

Results: - 38,848 residential houses were found out of 69,169 buildings in the area. - The filtered data is saved in the artifact: residential_houses_in_filtered_exeter

If you need further analysis or details about these houses, let me know!

Tool send_message returned result: The search for all houses in the area defined by 'filtered_exeter_polygon', filtered for residential houses only, has been completed.

Definition used for "house" (as per guidelines): - buildinguse_addresscount_residential > 0 - buildinguse_addresscount_total = 1 (This ensures only single-address residential houses are included.)

Results: - 38,848 residential houses were found out of 69,169 buildings in the area. - The filtered data is saved in the artifact: residential_houses_in_filtered_exeter

If you need further analysis or details about these houses, let me know!

Tool send_message called with args:

```
{'target': 'address', 'task_description': "Find all GP surgeries in the area defined by 'filtered_exeter_polygon'. Filter for GP surgeries only."}
```

address_agent

Tool call_os_ngd called with args:

```
{'filters': None, 'bbox': 'filtered_exeter_polygon', 'polygon_or_point': True, 'street_address': False, 'filename': 'gp_surgeries_exeter'}
```

Tool call_os_ngd returned result: - Search results have been found. Artifact generated is : gp_surgeries_exeter, Description is : A geopandas dataframe containing address data with filters and bbox applied for the query Find all GP surgeries in the area defined by 'filtered_exeter_polygon'. Filter for GP surgeries only. using filters None. Further Name filtering is available for this, count of records fetched : 75668.

Tool send_message called with args:

```
{'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'gp_surgeries_exeter' to include only GP surgeries. Save the filtered results as 'f"}
```

data_analysis_agent

Tool generate_metadata_for_artifacts called with args:

```
{'artifact_names': ['gp_surgeries_exeter']}
```

Tool generate_metadata_for_artifacts returned result.

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_gp_surgeries_exeter(data: list):
    import pandas as pd
    import geopandas as gpd
    gdf = data[0]
    # Filter for GP surgeries using relevant columns
    # We'll use 'organisationname', 'classificationdescription', 'primaryclassificationdescription', and 'quaternaryclassificationdescription'
    # Common terms for GP surgeries: 'GP', 'General Practice', 'Doctor', 'Medical Centre', 'Health Centre', 'Surgery'
    mask = (
        (gdf['organisationname'].str.contains('GP|General Practice|Doctor|Medical Centre|Health Centre|Surgery', case=False, na=False)) |
        (gdf['classificationdescription'].str.contains('Health', case=False, na=False)) |
        (gdf['primaryclassificationdescription'].str.contains('Health', case=False, na=False)) |
        (gdf['quaternaryclassificationdescription'].str.contains('Health', case=False, na=False))
    )
    filtered_gdf = gdf[mask]
    summary = f"Filtered to GP surgeries: {len(filtered_gdf)} entries found."
    artifact_name = 'filtered_gp_surgeries_exeter'
    artifact_description = 'GeoDataFrame of addresses in Exeter filtered to include only GP surgeries, using organisation and classification columns.'
    return [summary, artifact_name, artifact_description, filtered_gdf]
```

Filtered to GP surgeries: 27 entries found. A new artifact named 'filtered_gp_surgeries_exeter' has been created. This artifact is a GeoDataFrame of addresses in Exeter, filtered to include only GP surgeries, using organisation and classification columns for accurate selection.

Tool send_message returned result: Filtered to GP surgeries: 27 entries found. A new artifact named 'filtered_gp_surgeries_exeter' has been created. This artifact is a GeoDataFrame of addresses in Exeter, filtered to include only GP surgeries, using organisation and classification columns for accurate selection.

address_agent

I found 27 GP surgeries in the area defined by 'filtered_exeter_polygon'. The filtered results are saved in the artifact named 'filtered_gp_surgeries_exeter'.

Tool send_message returned result: I found 27 GP surgeries in the area defined by 'filtered_exeter_polygon'. The filtered results are saved in the artifact named 'filtered_gp_surgeries_exeter'.

host_agent

Tool send_message called with args:

```
{'target': 'plotting_agent', 'task_description': "Calculate and plot the distance from each house in 'residential_houses_in_filtered_exeter' to its closest GP"}
```

plotting_agent

Tool generate_metadata_for_artifacts called with args:

```
{'artifact_names': ['residential_houses_in_filtered_exeter', 'filtered_gp_surgeries_exeter']}
```

Tool generate_metadata_for_artifacts returned result.

Tool Code Executor (first attempt failed due to geometry type error)

```
import matplotlib
matplotlib.use('Agg')
def plot_house_to_gp_distances(data: list):
    import geopandas as gpd
    import pandas as pd
    import numpy as np
    from shapely.geometry import Point
    from shapely.ops import nearest_points
    import folium
    from folium.plugins import MarkerCluster
    import matplotlib.pyplot as plt

    # Unpack data
    houses = data[0].copy()
    gps = data[1].copy()

    # Ensure both are in EPSG:4326 for folium
    houses = houses.to_crs(epsg=4326)
    gps = gps.to_crs(epsg=4326)

    # Get centroids of houses (polygons)
    houses['centroid'] = houses.geometry.centroid

    # Build spatial index for GP surgeries
    gps_sindex = gps.sindex

    # Function to find nearest GP surgery and distance
    def nearest_gp(row, gps, gps_sindex):
        # Find nearest GP surgery
```

```

possible_matches_index = list(gps_sindex.nearest(row['centroid'].bounds, 1))
possible_matches = gps.iloc[possible_matches_index]
nearest_geom = possible_matches.geometry.iloc[0]
dist = row['centroid'].distance(nearest_geom)
return pd.Series({'nearest_gp_idx': possible_matches.index[0], 'distance_m': dist * 111139}) # approx meters

# Calculate nearest GP and distance for each house
houses[['nearest_gp_idx', 'distance_m']] = houses.apply(lambda row: nearest_gp(row, gps, gps_sindex), axis=1)

# Merge GP info for popup
houses = houses.merge(gps.reset_index()[['index', 'fulladdress']], left_on='nearest_gp_idx', right_on='index', how='left', suffixes=('', '_gp'))

# Distance summary
dist_summary = houses['distance_m'].describe(percentiles=[.05, .25, .5, .75, .95]).to_dict()
dist_summary_str = '\n'.join([f"{k}: {v:.2f}" for k, v in dist_summary.items()])

# Folium map
m = folium.Map(location=[houses['centroid'].y.mean(), houses['centroid'].x.mean()], zoom_start=12)

# Add GP surgeries
for idx, row in gps.iterrows():
    folium.Marker(
        location=[row.geometry.y, row.geometry.x],
        popup=f"GP: {row.get('name', 'Unknown')}<br>{row['fulladdress']}",
        icon=folium.Icon(color='red', icon='plus-sign')
    ).add_to(m)

# Add houses (sample for performance)
sample_houses = houses.sample(n=min(200, len(houses)), random_state=42)
for idx, row in sample_houses.iterrows():
    folium.CircleMarker(
        location=[row['centroid'].y, row['centroid'].x],
        radius=3,
        color='blue',
        fill=True,
        fill_opacity=0.5,
        popup=f"House: {row.get('description', 'House')}<br>Distance to nearest GP: {row['distance_m']:.1f} m<br>Nearest GP: {row['fulladdress']}"
    ).add_to(m)

# Draw line to nearest GP
gp_row = gps.loc[row['nearest_gp_idx']]
folium.PolyLine(
    locations=[
        [row['centroid'].y, row['centroid'].x],
        [gp_row.geometry.y, gp_row.geometry.x]
    ],
    color='green',
    weight=1
).add_to(m)

# Save map

```

```

map_filename = 'house_to_gp_distances_map.html'
m.save(map_filename)

# Prepare summary
summary = f"Calculated distances from {len(houses)} houses to their nearest GP surgery.\n" \
          f"Distance distribution (meters):\n{dist_summary_str}\n" \
          f"First 5 houses:\n" + str(houses[['osid', 'description', 'distance_m', 'fulladdress']].head().to_dict('records'))

return [summary, map_filename, "Folium map showing houses, GP surgeries, and lines to nearest GP. Distance distribution summary included.", map_filename]

```

Tool Code Executor (second attempt, correct version)

```

import matplotlib
matplotlib.use('Agg')
def plot_house_to_gp_distances(data: list):
    import geopandas as gpd
    import pandas as pd
    import numpy as np
    from shapely.geometry import Point
    import folium

    houses = data[0].copy()
    gps = data[1].copy()

    # Ensure both are in EPSG:4326 for folium
    houses = houses.to_crs(epsg=4326)
    gps = gps.to_crs(epsg=4326)

    # Get centroids of houses (polygons)
    houses['centroid'] = houses.geometry.centroid

    # Prepare a GeoSeries of GP points
    gp_points = gps.geometry

    # For each house, find the nearest GP and distance
    nearest_gp_idx = []
    nearest_gp_dist = []
    for centroid in houses['centroid']:
        dists = gp_points.distance(centroid)
        min_idx = dists.idxmin()
        nearest_gp_idx.append(min_idx)
        # Use geodesic approx: 1 deg lat ~ 111139 m
        nearest_gp_dist.append(dists[min_idx] * 111139)
    houses['nearest_gp_idx'] = nearest_gp_idx
    houses['distance_m'] = nearest_gp_dist

    # Merge GP info for popup

```

```

houses = houses.merge(gps.reset_index()[['index', 'fulladdress']], left_on='nearest_gp_idx', right_on='index', how='left', suffixes=('', '_gp'))

# Distance summary
dist_summary = houses['distance_m'].describe(percentiles=[.05, .25, .5, .75, .95]).to_dict()
dist_summary_str = '\n'.join([f"{k}: {v:.2f}" for k, v in dist_summary.items()])

# Folium map
m = folium.Map(location=[houses['centroid'].y.mean(), houses['centroid'].x.mean()], zoom_start=12)

# Add GP surgeries
for idx, row in gps.iterrows():
    folium.Marker(
        location=[row.geometry.y, row.geometry.x],
        popup=f"GP: {row.get('name', 'Unknown')}<br>{row['fulladdress']}",
        icon=folium.Icon(color='red', icon='plus-sign')
    ).add_to(m)

# Add houses (sample for performance)
sample_houses = houses.sample(n=min(200, len(houses)), random_state=42)
for idx, row in sample_houses.iterrows():
    folium.CircleMarker(
        location=[row['centroid'].y, row['centroid'].x],
        radius=3,
        color='blue',
        fill=True,
        fill_opacity=0.5,
        popup=f"House: {row.get('description', 'House')}<br>Distance to nearest GP: {row['distance_m']:.1f} m<br>Nearest GP: {row['fulladdress']}"
    ).add_to(m)

# Draw line to nearest GP
gp_row = gps.loc[row['nearest_gp_idx']]
folium.PolyLine(
    locations=[
        [row['centroid'].y, row['centroid'].x],
        [gp_row.geometry.y, gp_row.geometry.x]
    ],
    color='green',
    weight=1
).add_to(m)

# Save map
map_filename = 'house_to_gp_distances_map.html'
m.save(map_filename)

# Prepare summary
summary = f"Calculated distances from {len(houses)} houses to their nearest GP surgery.\n" \
    f"Distance distribution (meters):\n{dist_summary_str}\n" \
    f"First 5 houses:\n" + str(houses[['osid', 'description', 'distance_m', 'fulladdress']].head().to_dict('records'))

return [summary, map_filename, "Folium map showing houses, GP surgeries, and lines to nearest GP. Distance distribution summary included.", map_filename]

```